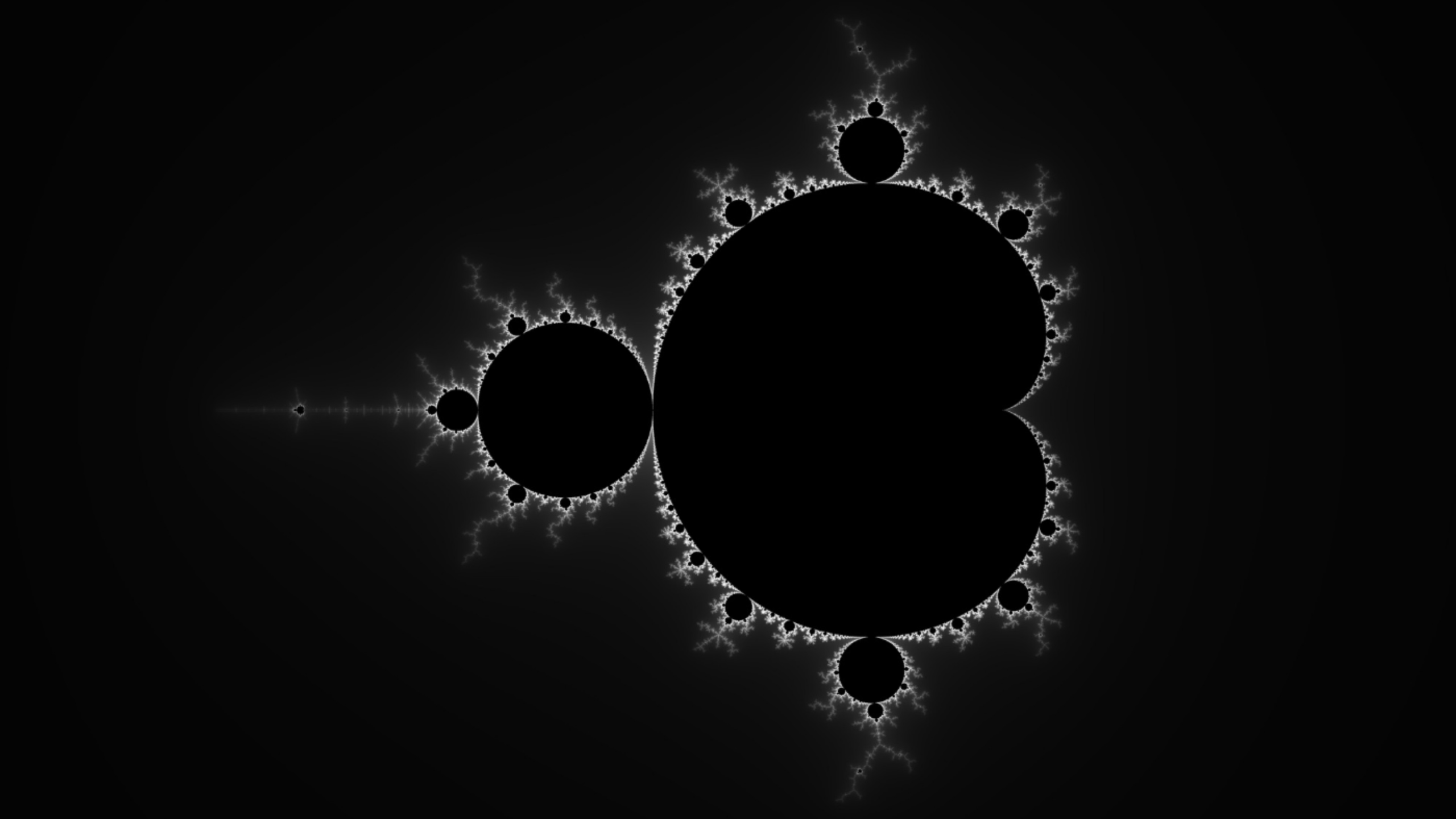
Programação Paralela · Trabalho SO2

Mandelbrot PPM

Sequencial, múltiplos processos com memória compartilhada e múltiplas threads

Elton Giglio Kasesky
Frederico Bazano



Problema original

O que é

Renderizar o fractal de Mandelbrot em 8K (7680×4320, 33,2 milhões de pixels) e gravar em PPM (P6). Cada pixel itera $z = z^2 + c$ até escapar ou chegar a 1500 iterações.

Entrada

factor = 480 → resolução 16·factor × 9·factor. Linguagem: C (gcc -O3).

Saída

Imagem PPM em tons de cinza (94,92 MB) e métricas: tempo de cálculo, de I/O e total de iterações.

7,5 bi

iterações de Mandelbrot na carga de teste

~17 s

de cálculo na versão sequencial

Versão sequencial

Funcionamento

Dois laços (y e x) chamam pixelShader(x, y) e preenchem o framebuffer. Depois writeImage grava o arquivo. O tempo de cálculo e o de I/O são medidos separadamente.

17,14 s
cálculo

0,60 s
gravação (I/O)

17,75 s
tempo total

O cálculo é 96,6% do total. A gravação continua sequencial nas versões paralelas e limita o speedup.

Estratégia de paralelização

Divisão por linhas

Cada pixel é independente. A linha y é calculada pelo trabalhador p quando y % N == p (round-robin).

Por que intercalar

Linhas do centro custam até 1500 iterações e as das bordas custam poucas. Intercalar espalha o custo entre todos sem fila de tarefas.

N na execução

O número de processos ou threads é argumento: ./bin/processos N arquivo factor. Nada fixo no código.

Implementação com processos

Memória compartilhada

mmap com MAP_SHARED | MAP_ANONYMOUS cria o framebuffer (94,92 MB) e a estrutura SharedState, visíveis a todos os filhos.

fork()

O pai cria N filhos. Cada filho grava direto nos pixels das suas linhas, sem cópia nem pipe.

Pai

waitpid espera todos os filhos, mede o tempo, grava a imagem e libera a memória com munmap.

Sincronização

Processos

sem_t com pshared = 1, dentro do SharedState. sem_wait / sem_post protegem total_iterations e workers_completed.

Threads

pthread_mutex_lock / unlock protegem o contador total_iterations. A espera final é pthread_join.

Região crítica mínima

Cada trabalhador acumula iterações numa variável local e entra na região crítica uma única vez, no fim.

Pixels sem lock

Linhas disjuntas por trabalhador: nenhuma posição do framebuffer é escrita por dois ao mesmo tempo.

Implementação com threads

Mesmo espaço de endereço

As threads compartilham o framebuffer alocado com malloc. Não precisa de mmap.

pthread_create

N threads recebem uma struct ThreadData (id, N, dimensões, ponteiros). A divisão de linhas é igual à dos processos.

Processos x threads

Mesmo algoritmo e mesma divisão. Muda o mecanismo: fork + mmap + semáforo contra pthread + mutex.

Resultados experimentais

Gráfico 1 · Tempo de cálculo (fora do container)

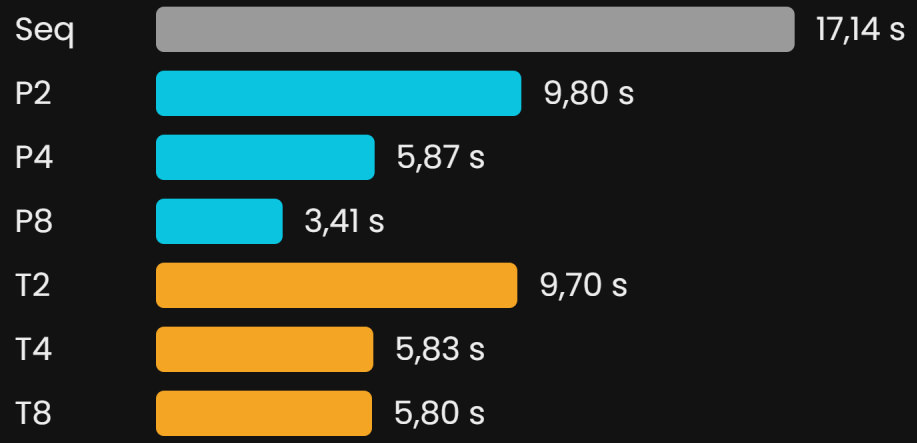
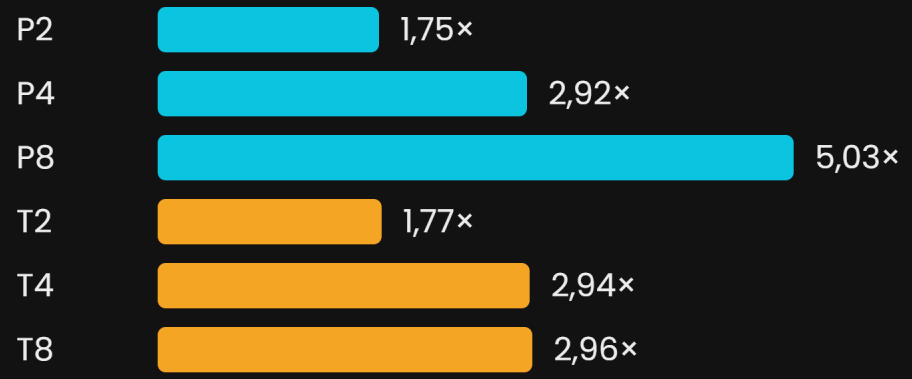


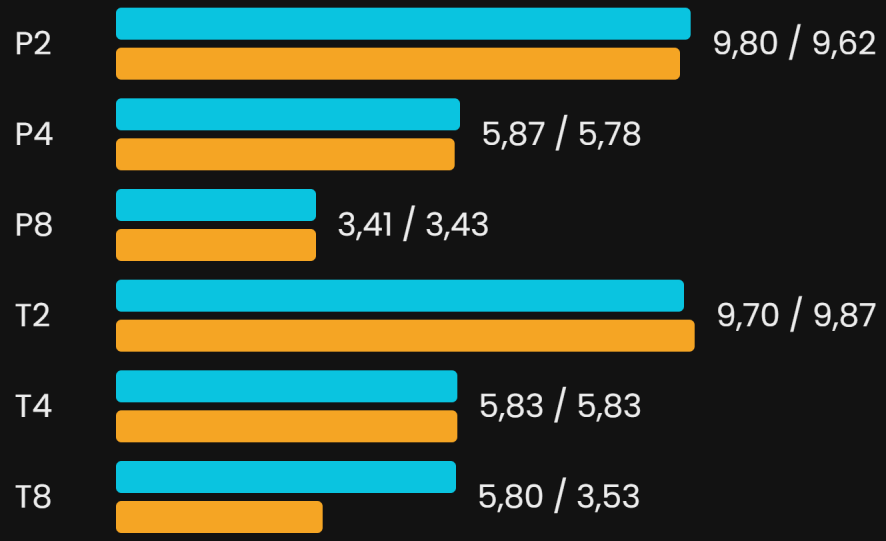
Gráfico 2 · Speedup sobre o sequencial



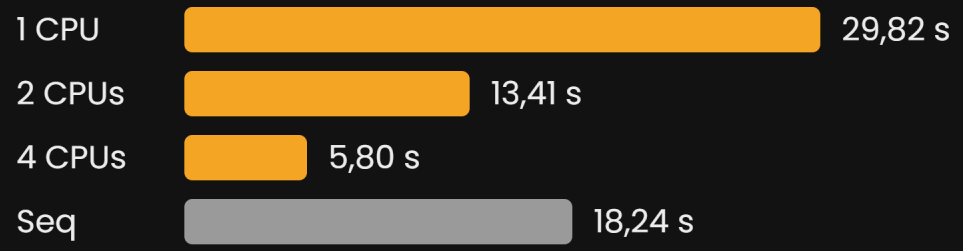
P = processos (ciano), T = threads (laranja). Speedup = 17,14 s ÷ tempo. Com 8 threads fora do container: 5,80 s; dentro do container: 3,53 s.

Container Docker e limite de CPU

Gráfico 3 · Fora (ciano) × dentro (laranja), em s



Processos (N=4) com --cpus



Sem limite, fora e dentro do container diferem menos de 3%, exceto em 8 threads (5,80 s fora, 3,53 s dentro). Com --cpus=1 a versão com 4 processos fica mais lenta que o sequencial.

Comparação entre as três versões

N	Processos	Threads
2	9,80 s · 1,75×	9,70 s · 1,77×
4	5,87 s · 2,92×	5,83 s · 2,94×
8	3,41 s · 5,03×	5,80 s · 2,96×

As três versões calculam exatamente 7.514.945.309 iterações, logo os resultados são equivalentes. Base sequencial: 17,14 s.

Conclusões

Speedup sublinear

Eficiência de 88% com 2, 73% com 4 e 63% com 8 trabalhadores. Causas prováveis: I/O sequencial (0,6 s) e núcleos lógicos compartilhados.

Processos ≈ threads

Com 2 e 4 trabalhadores a diferença é menor que 1,5%. Com 8, as threads fora do container (5,80 s) ficaram atrás dos processos (3,41 s).

Limite de CPU manda

Com 4 CPUs o container iguala o desempenho nativo. Com 1 CPU, 4 processos levam 29,8 s: o limite do container pesa mais que o número de processos.